

5-1장 과제

2101080 정진용

- 빈파일을 만드는 명령어는 무엇인가? touch입니다
- 디렉터리를 만드는 명령어는 무엇인가? mkdir입니다
- 홈디렉터리 안에 다음과 같이 디렉터리 dir1, dir2, dir3를 만들고 각각 디렉터리 안에 다음과 같이 빈파일을 작성시오.

```
Windows PowerShell  x  linux@localhost: ~  +  v  -  □  X

linux@localhost:~$ mkdir dir1 dir2 dir3
linux@localhost:~$ touch ./dir1/file1 ./dir1/file2 ./dir1/file3
linux@localhost:~$ touch ./dir2/newfile1 ./dir2/newfile2 ./dir2/newfile3
linux@localhost:~$ touch ./dir3/oldfile1 ./dir3/oldfile2 ./dir3/oldfile3
linux@localhost:~$ ls ./dir1 -F
file1 file2 file3
linux@localhost:~$ ls ./dir2 ./dir3 -F
./dir2:
newfile1 newfile2 newfile3

./dir3:
oldfile1 oldfile2 oldfile3
linux@localhost:~$
```

- tree 명령어를 이용하면 파일목록을 트리구조로 출력해줌

```
.
├── dir1
│   ├── file1
│   ├── file2
│   └── file3
├── dir2
│   ├── newfile1
│   ├── newfile2
│   └── newfile3
└── dir3
    ├── oldfile1
    ├── oldfile2
    └── oldfile3
```

□ 빈 디렉터리를 지우는 명령어는 무엇인가? rmdir입니다

□ 비어 있지 않는 디렉터를 지우는 명령어는 무엇인가? rm -r [디렉터리]

□ 파일을 지우는 명령어는 무엇인가? rm

□ 실습과제 1번에서 만든 디렉터리와 파일을 모두 삭제하시오

□ 삭제할 때 경로확장 기능을 사용할 것

```
linux@localhost:~$ rm -r *  
linux@localhost:~$ ls  
linux@localhost:~$
```

5-2장 과제

실습과제1

□ 홈디렉터리 안에 다음과 같이 디렉터리 dir1, dir2, dir3를 만들고 각각 디렉터리 안에 다음과 같이 빈파일을 작성하시오.

□ tree 명령어를 이용하면 파일목록을 트리구조로 출력해줌

```
linux@localhost:~$ mkdir dir1 dir2 dir3
linux@localhost:~$ cd dir1
linux@localhost:~/dir1$ touch file1 file2 file3
linux@localhost:~/dir1$ cd ../dir2
linux@localhost:~/dir2$ touch newfile1 newfile2 newfile3
linux@localhost:~/dir2$ cd ../dir3
linux@localhost:~/dir3$ touch oldfile1 oldfile2 oldfile3
linux@localhost:~/dir3$ tree
.
├── oldfile1
├── oldfile2
└── oldfile3

0 directories, 3 files
linux@localhost:~/dir3$ cd ../
linux@localhost:~$ tree
.
├── dir1
│   ├── file1
│   ├── file2
│   └── file3
├── dir2
│   ├── newfile1
│   ├── newfile2
│   └── newfile3
└── dir3
    ├── oldfile1
    ├── oldfile2
    └── oldfile3

3 directories, 9 files
linux@localhost:~$
```

실습과제2

□ cp명령어를 이용하여 실습과제1에서 만든 3개의 디렉토리를 다음처럼 복사하시오 ->
경로확장기능을 이용하여 복사할 것

```
linux@localhost:~$ mkdir cpdir1 cpdir2 cpdir3
linux@localhost:~$ cp dir1/file? cpdir1
linux@localhost:~$ cp dir2/newfile? cpdir2
linux@localhost:~$ cp dir3/oldfile? cpdir3
linux@localhost:~$ tree
```

```
.
├── cpdir1
│   ├── file1
│   ├── file2
│   └── file3
├── cpdir2
│   ├── newfile1
│   ├── newfile2
│   └── newfile3
├── cpdir3
│   ├── oldfile1
│   ├── oldfile2
│   └── oldfile3
├── dir1
│   ├── file1
│   ├── file2
│   └── file3
├── dir2
│   ├── newfile1
│   ├── newfile2
│   └── newfile3
└── dir3
    ├── oldfile1
    ├── oldfile2
    └── oldfile3
```

```
6 directories, 18 files
linux@localhost:~$
```

실습과제3

□ mv명령어를 이용하여 실습과제1에서 만든 3개의 디렉토리안의 파일들을 다음처럼 새로운 디렉토리를 만들고 파일들을 새이름으로 이동하시오 -> 원본이 있는 디렉토리에서는 삭제되는지 확인할것

```
linux@localhost:~$ mv cpdir1/file1 mvdir1/mvfile1
linux@localhost:~$ mv cpdir1/file2 mvdir1/mvfile2
linux@localhost:~$ mv cpdir1/file3 mvdir1/mvfile3
linux@localhost:~$ mv cpdir2/newfile1 mvdir2/mvnewfile1
linux@localhost:~$ mv cpdir2/newfile2 mvdir2/mvnewfile2
linux@localhost:~$ mv cpdir2/newfile3 mvdir2/mvnewfile3
linux@localhost:~$ mv cpdir3/oldfile1 mvdir3/mvnewfile1
linux@localhost:~$ mv cpdir3/oldfile2 mvdir3/mvnewfile2
linux@localhost:~$ mv cpdir3/oldfile3 mvdir3/mvnewfile3
linux@localhost:~$ tree
```

```
.
├── cpdir1
├── cpdir2
├── cpdir3
├── dir1
│   ├── file1
│   ├── file2
│   └── file3
├── dir2
│   ├── newfile1
│   ├── newfile2
│   └── newfile3
├── dir3
│   ├── oldfile1
│   ├── oldfile2
│   └── oldfile3
├── mvdir1
│   ├── mvfile1
│   ├── mvfile2
│   └── mvfile3
├── mvdir2
│   ├── mvnewfile1
│   ├── mvnewfile2
│   └── mvnewfile3
└── mvdir3
    ├── mvnewfile1
    ├── mvnewfile2
    └── mvnewfile3
```

9 directories, 18 files

```
linux@localhost:~$
```

5-3장 과제

□ 하드링크와 심볼릭 링크의 차이를 설명하라.

하드링크는 원본 파일의 또 다른 이름으로, 실제 파일 데이터와 직접 연결됩니다. 같은 파일의 다른 경로로 생각할 수 있습니다.

하드링크는 원본 파일과 동일한 inode 번호를 공유하며, 파일의 데이터는 동일합니다.

하드링크는 원본 파일을 삭제하더라도 하드링크는 여전히 파일에 접근할 수 있습니다.

파일의 데이터는 하드링크가 남아 있는 한 삭제되지 않습니다.

하드링크는 같은 파일 시스템 내에서만 생성할 수 있습니다.

심볼릭 링크는 원본 파일의 경로를 가리키는 파일입니다. 원본 파일 자체가 아니라 원본 파일의 위치를 참조합니다.

심볼릭 링크는 원본 파일과 다른 indode 번호를 가집니다

심볼릭 링크원본 파일이 삭제되면 심볼릭 링크는 더 이상 유효하지 않으며, 깨진 링크가 됩니다.

심볼릭 링크는 다른 파일 시스템 간에도 생성할 수 있습니다.

□ 심볼릭 링크를 확인하는 명령어를 설명하라

ls -l입니다. ls 명령어를 실행할 때 -l 옵션을 지정하면 심볼릭 링크 여부를 확인할 수 있습니다.

□ 실제로 심볼릭 링크를 만들고 확인한 결과를 첨부하라.

```
linux@localhost:~$ cp /etc/crontab file1
linux@localhost:~$ ln -s file1 file2
linux@localhost:~$ ls
file1  file2
linux@localhost:~$ cat file2
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .----- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
linux@localhost:~$
```